{\rtf1\ansi\ansicpg1252\deff0\nouicompat\deflang1036{\fonttbl{\f0\fnil\fcharset0 Calibri;}{\f1\fnil Calibri;}}
{*\generator Riched20 10.0.22621}\viewkind4\uc1
\pard\sa200\sl276\slmult1\f0\fs22\lang9 Cover Page\par
Title:\par
Comparison Between Non-RDBMS and RDBMS\par
\par
Course Title:\par
\par
Prepared by:\par
\par
Submitted to:\par
\par
Submission Date:\par
\par
Title:\par
Comparative Analysis of Non-RDBMS and RDBMS\par
\par
Introduction\par
Relational Database Management Systems (RDBMS) have been the cornerstone of data storage for decades, supporting a wide range of applications. With the advent of big data and increasing diversity in data types, Non-Relational Database Management Systems (Non-RDBMS) or NoSQL databases have emerged as a viable alternative for certain use cases. This research aims to compare the fundamental differences between RDBMS and Non-RDBMS, outlining their structures, benefits, and limitations.\par
\par
Body\par
1. Definition and Overview\par
RDBMS:\par
\par
Relational Database Management Systems (RDBMS) are databases based on a structured model of data where information is stored in tables with predefined schema. Data is organized in rows and columns, and relationships between tables are established using keys. Examples include MySQL, PostgreSQL, and Oracle.\par
\par
Non-RDBMS:\par
\par
Non-RDBMS, commonly known as NoSQL (Not Only SQL), refers to a broad category of databases designed to handle unstructured or semi-structured data. These systems allow for more flexibility in terms of data modeling and are optimized for horizontal scaling. Examples include MongoDB, Cassandra, and Couchbase.\par
\par
2. Data Structure and Storage\par
RDBMS:\par
\par

Data is stored in a tabular format with a fixed schema.\par
Enforces relationships through primary and foreign keys.\par
Ideal for applications that require ACID (Atomicity, Consistency, Isolation, Durability) properties.\par
Examples: MySQL, PostgreSQL, Oracle.\par
Non-RDBMS:\par
\par
Data can be stored in various formats such as key-value pairs, documents, graphs, or wide-column stores.\par
It allows for schema-less designs, meaning the data structure can evolve over time.\par
More suited for applications that require flexibility in data storage and retrieval.\par
Examples: MongoDB (document-based), Cassandra (wide-column), Neo4j (graph-based).\par
3. Scalability\par
RDBMS:\par
\par
Traditionally supports vertical scaling, which means upgrading the hardware (CPU, RAM) to manage larger loads.\par
Scaling horizontally (adding more servers) is more complex.\par
Non-RDBMS:\par
\par
Designed for horizontal scaling, which allows for easy distribution of data across multiple servers.\par
Particularly effective for large-scale applications with distributed data storage needs.\par
4. Flexibility and Schema Design\par
RDBMS:\par
\par
Relies on a strict schema design that defines the structure of data before any information is entered.\par
Ideal for structured data where relationships and constraints are crucial.\par
Non-RDBMS:\par
\par
Offers greater flexibility with schema-less or dynamic schema designs.\par
Data can be added without predefined structures, making it suitable for rapidly changing or unstructured data like social media content.\par
5. Performance\par
RDBMS:\par
\par
Performs well for read and write operations in structured datasets but may suffer performance degradation when handling massive, complex queries.\par
High consistency due to ACID properties.\par
Non-RDBMS:\par
\par
Optimized for large-scale, unstructured data with quick reads and writes.\par
Offers eventual consistency, meaning that changes in the database propagate

over time, but not all nodes may reflect those changes instantly.\par
6. Use Cases\par
RDBMS:\par
\par
Ideal for transactional applications like banking systems, e-commerce websites, and enterprise-level applications where data integrity and consistency are critical.\par
Strongly suited for structured data with clear relationships.\par
Non-RDBMS:\par
\par
Suitable for big data applications, real-time analytics, content management systems, and applications with dynamic or unstructured data.\par
Examples include social media platforms, IoT applications, and recommendation engines.\par
Conclusion\par
While RDBMS remains a robust solution for structured data with a strong emphasis on consistency and relationships, Non-RDBMS provides flexibility, scalability, and performance advantages for unstructured or semi-structured data. The choice between RDBMS and Non-RDBMS should be guided by the specific needs of the application, particularly in terms of data structure, scalability, and performance requirements.\par
\par
References\par
Stonebraker, M., & Hellerstein, J. M. (2005). What goes around comes around. Readings in Database Systems. MIT Press.\par
Han, J., Song, M., & Eom, J.-H. (2011). A survey on NoSQL databases. Proceedings of the 2011 IEEE Conference on Cloud and Service Computing (CSC), 40\f1\endash 47.\par
Sadalage, P., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.\f0\par
}